

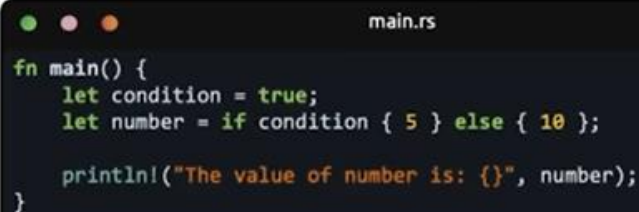
```

if number > 1 && number < 8 {
    println!("Number Between 1 and 8");
}
*****

```

Using "if" in a "let" Statement

Example



```

fn main() {
    let condition = true;
    let number = if condition { 5 } else { 10 };

    println!("The value of number is: {}", number);
}

```

```

*****

let mut count: i32 = 0;

loop {
    count += 1;
    println!("The count is: {}", count);
    if count == 6 {
        break;
    }
}
*****

```

We can return value from loop:

```

let result = loop {
    count += 1;
    println!("The count is: {count}");

    if count > 6 {
        break count * 2;
    }
};

println!("The final result is: {result}");
*****

```

The result is mut-variable:

```
'outer: loop {  
  
    loop {  
        if result > 15 {  
            break 'outer;  
        }  
        if result > 13 {  
            break;  
        }  
    }  
    result += 1;  
    println!("The result is: {result}");  
}
```

In Rust, the continue statement works similarly to how it does in other languages like C or Python. It is used to skip the current iteration of a loop and immediately jump to the next iteration, without completing the remaining statements in the current loop body.

The "for" Loop

The for loop in Rust is used to iterate over a collection of elements such as arrays, vectors, ranges, and iterators.

Syntax

```
for variable in collection {  
    // code to run for each element in the collection  
}
```

Pattern Matching With "match"

Pattern matching with the match statement in Rust is a powerful control flow construct that allows you to compare a value against a series of patterns and execute code based on which pattern matches.





01

The match statement in Rust is **like switch statements** in other languages but is much more powerful.

02

It can handle a variety of patterns, including **literals, variables, wildcards**, and more complex patterns like **tuples** and **enums**.

03

Using match, we can **write concise and expressive code** that clearly expresses your intent.

```
let mut number = 3;

while number != 0 {
    println!("number is: {number}");
    number -= 1;
}

println!("Hello, world!");
```

```

let arr: [i32; 5] = [1, 2, 3, 4, 5];

for num in arr {
    println!("The num is: {num}");
}

for i in 1..6 {
    println!("i is: {i}");
}
*****

```

OR operator (|)

In Rust, the OR operator (|) is used to specify multiple patterns in a match expression. It allows you to match multiple values to the same arm of the match block. This is helpful when you want to execute the same logic for multiple possible values.

Example



```

1 | 2 | 3 => println!("The number is 1, 2,
or 3");

```

```

let number: i32 = 3;
match number {
    1 => println!("The number is 1"),
    2 => println!("The number is 2"),
    3 => println!("The number is 3"),
    _ => println!("Something else"),
}

let number: i32 = 7;
match number {
    1..=5 => println!("Number between 1 and 5"),
    6..=10 => println!("Number between 6 and 10"),
    _ => println!("Something else"), // if we don't set it, it will give
error
}

println!("Hello, world!");

```

Destructuring With "match"

Example



```

fn main() {
    let pair = (2, -2);

    match pair {
        (x, y) if x == y => println!("The numbers are equal"),
        (x, y) if x + y == 0 => println!("The numbers are
opposites"),
        _ => println!("No special properties"),
    }
}

```

This example about using wildcard `_`

Ignoring Values in Patterns

Example

```
main.rs

fn main() {
    let pair = (2, 8);

    match pair {
        (x, _) => println!("The first number is: {}", x),
    }
}
```